

CESIZen

Plateforme de Gestion du Stress & Suivi Émotionnel

Cahier de Tests, Étude Comparative & Guide d'Installation

Commanditaire : Ministère des Solidarités et de la Santé

Bloc : Concevoir les solutions logicielles (INFCDAAL1)

École : CESI École d'Ingénieurs

Date : Mars 2026

Version : 1.0

Table des matières

1	Cahier de Tests & Critères d'Évaluation	3
1.1	Critères d'évaluation techniques	3
1.2	Tests fonctionnels - Authentification	3
1.3	Tests fonctionnels - Tracker émotionnel	4
1.4	Tests fonctionnels - Feeds / Ressources	4
1.5	Tests fonctionnels - Administration	5
1.6	Tests fonctionnels - Profil utilisateur	5
1.7	Tests non fonctionnels	5
2	Tableau Comparatif Détaillé	6
2.1	Solutions comparées	6
2.2	Comparaison par catégorie	7
3	Synthèse des Scores Pondérés	8
3.1	Matrice de pondération	8
3.2	Résultats finaux	8
4	Pertinence de la Solution Retenue	9
4.1	Solution idéale identifiée	9
4.2	Justification technique	9
4.3	Adéquation aux besoins	10
5	Guide d'Installation	10
5.1	Prérequis	10
5.2	Installation avec Docker	11
5.3	Installation manuelle	11
5.4	Configuration	12
6	Procédures de Validation	12
6.1	Plan de validation	12
6.2	Checklist de validation	13
6.3	Critères d'acceptation	13
6.4	Modèle de procès-verbal de recette	14

1. Cahier de Tests & Critères d'Évaluation

1.1 Critères d'évaluation techniques

Les critères ci-dessous définissent les exigences qualité du projet CESIZen. Chaque critère est noté de 1 (insuffisant) à 5 (excellent) et pondéré selon son importance.

Critère	Pondération	Description	Seuil min.
Performance	15%	Temps de réponse API < 200ms	3/5
Sécurité	20%	Auth JWT, CORS, validation, RGPD	4/5
Maintenabilité	15%	Architecture MVC, code documenté	3/5
Scalabilité	10%	Docker, services découplés	3/5
Ergonomie (UX)	15%	Interface intuitive, responsive	3/5
Conformité fonctionnelle	15%	Couverture des user stories	4/5
Fiabilité	10%	Gestion d'erreurs, logs, audit	3/5

Outils de test automatisés

Les tests sont automatisés sur les trois briques du projet et exécutables en une commande :

- Backend (API Laravel) : PHPUnit - 15 tests (authentification, tracker, admin, RGPD). Commande : php artisan test.
- Frontend Web (Next.js) : Vitest + Testing Library - composants UI et store d'authentification. Commande : npm test.
- Mobile (React Native / Expo) : Jest (preset jest-expo) - utilitaires d'appel API. Commande : npm test.

1.2 Tests fonctionnels - Authentification

ID	Cas de test	Entrée	Résultat attendu	Statut	Responsable
T-AUTH-01	Inscription valide	Nom, prénom, email, mdp	Compte créé, token JWT	OK	Dév. backend
T-AUTH-02	Inscription email existant	Email déjà utilisé	Erreur 422, msg clair	OK	Dév. backend
T-AUTH-03	Connexion valide	Email + mdp corrects	Token JWT retourné	OK	Dév. backend
T-AUTH-04	Connexion mdp erroné	Email + mdp incorrect	Erreur 401	OK	Dév. backend
T-AUTH-05	Connexion compte inactif	Compte désactivé	Erreur 403	OK	Dév. backend
T-AUTH-06	Rate limiting login	6 tentatives en 1 min	Erreur 429 throttle	OK	Dév. backend
T-AUTH-07	Déconnexion	Token valide	Token invalidé	OK	Dév. backend
T-AUTH-08	Refresh token	Token expirant	Nouveau token retourné	OK	Dév. backend
T-AUTH-09	Accès route protégée sans token	Aucun token	Erreur 401	OK	Dév. backend
T-AUTH-10	Accès route admin sans rôle	Token membre	Erreur 403	OK	Dév. backend

1.3 Tests fonctionnels - Tracker émotionnel

ID	Cas de test	Entrée	Résultat attendu	Statut	Responsable
----	-------------	--------	------------------	--------	-------------

ID	Cas de test	Entrée	Résultat attendu	Statut	Responsable
T-TRK-01	Créer saisie	Émotion, intensité 1-10, note	Saisie enregistrée	OK	Dév. backend
T-TRK-02	Créer saisie sans tracker	Premier usage	Tracker auto-créé	OK	Dév. backend
T-TRK-03	Lister saisies	Token valide	Liste paginée (20/page)	OK	Dév. backend
T-TRK-04	Filtrer par date	date_debut, date_fin	Saisies dans la période	OK	Dév. backend
T-TRK-05	Filtrer par émotion	emotion_id	Saisies correspondantes	OK	Dév. backend
T-TRK-06	Modifier saisie	Nouvelle intensité/note	Saisie mise à jour	OK	Dév. backend
T-TRK-07	Supprimer saisie	ID saisie existante	Saisie supprimée	OK	Dév. backend
T-TRK-08	Rapport semaine	period=week	Stats hebdomadaires	OK	Dév. backend
T-TRK-09	Rapport mois	period=month	Stats mensuelles	OK	Dév. backend
T-TRK-10	Rapport trimestre	period=quarter	Stats trimestrielles	OK	Dév. backend
T-TRK-11	Rapport année	period=year	Stats annuelles	OK	Dév. backend
T-TRK-12	Saisie intensité hors bornes	intensité = 15	Erreur validation 422	OK	Dév. backend

1.4 Tests fonctionnels - Feeds / Ressources

ID	Cas de test	Entrée	Résultat attendu	Statut	Responsable
T-FEED-01	Lister feeds publiés	Aucun (public)	Liste ordonnée des articles	OK	Dév. backend
T-FEED-02	Détail feed par slug	Slug existant	Contenu complet + auteur	OK	Dév. backend
T-FEED-03	Feed slug inexistant	Slug invalide	Erreur 404	OK	Dév. backend
T-FEED-04	Feed non publié inaccessible	Feed draft	Non listé / 404	OK	Dév. backend

1.5 Tests fonctionnels - Administration

ID	Cas de test	Entrée	Résultat attendu	Statut	Responsable
T-ADM-01	Lister utilisateurs	Token admin	Liste paginée (15/p)	OK	Dév. backend
T-ADM-02	Rechercher utilisateur	search=nom	Résultats ilike	OK	Dév. backend
T-ADM-03	Créer utilisateur	Données complètes	Utilisateur + Tracker	OK	Dév. backend
T-ADM-04	Modifier utilisateur	Nouvelles données	Utilisateur mis à jour	OK	Dév. backend
T-ADM-05	Activer/Désactiver	ID user	Toggle est_actif	OK	Dév. backend
T-ADM-06	Supprimer utilisateur	ID user	Soft delete	OK	Dév. backend
T-ADM-07	CRUD feeds (admin)	Données feed	Création/modif/suppr.	OK	Dév. backend
T-ADM-08	CRUD émotions	Données émotion	Création/modif/désact.	OK	Dév. backend
T-ADM-09	Désactiver émotion parent	ID parent	Parent + enfants désact.	OK	Dév. backend
T-ADM-10	Audit trail	Actions admin	Logs en base (audits)	OK	Dév. backend

1.6 Tests fonctionnels - Profil utilisateur

ID	Cas de test	Entrée	Résultat attendu	Statut	Responsable
T-PRO-01	Voir profil	Token valide	Infos profil complètes	OK	Dév. backend
T-PRO-02	Modifier nom/prénom	Nouvelles valeurs	Profil mis à jour	OK	Dév. backend
T-PRO-03	Changer mot de passe	Ancien + nouveau mdp	Mot de passe changé	OK	Dév. backend
T-PRO-04	Ancien mdp incorrect	Mauvais ancien mdp	Erreur 422	OK	Dév. backend
T-PRO-05	Supprimer compte (RGPD)	Confirmation	Soft delete du compte	OK	DPO / Dév.

1.7 Tests non fonctionnels

ID	Catégorie	Test	Critère	Statut	Responsable
T-NF-01	Performance	Temps de réponse API	< 200ms (95e centile)	OK	DevOps
T-NF-02	Performance	Chargement page front	< 2s (First Contentful Paint)	OK	DevOps
T-NF-03	Sécurité	Injection SQL	Eloquent ORM paramétré	OK	Dév. backend

ID	Catégorie	Test	Critère	Statut	Responsable
T-NF-04	Sécurité	XSS	Échappement React natif	OK	Dév. front
T-NF-05	Sécurité	CORS	Origines restreintes	OK	Dév. backend
T-NF-06	Sécurité	JWT expiration	Tokens à durée limitée	OK	Dév. backend
T-NF-07	Sécurité	Rate limiting	Throttle 5 tentatives/min	OK	Dév. backend
T-NF-08	Accessibilité	Responsive design	Mobile, tablette, desktop	OK	Dév. front
T-NF-09	Accessibilité	Mode sombre	Toggle dark/light mode	OK	Dév. front
T-NF-10	RGPD	Droit à l'effacement	Soft delete + anonymisation	OK	DPO
T-NF-11	RGPD	Consentement	Champ consentement_rgpd	OK	DPO
T-NF-12	Fiabilité	Gestion erreurs API	Réponses JSON structurées	OK	DevOps
T-NF-13	Fiabilité	Audit trail admin	Table audits traçabilité	OK	DevOps

2. Tableau Comparatif Détaillé

2.1 Solutions comparées

Quatre architectures techniques ont été évaluées pour répondre aux besoins de la plateforme CESIZen. La comparaison porte sur des critères techniques, organisationnels et économiques.

Solution A : Laravel 12 + Next.js 16 (Solution retenue)

Backend API REST Laravel 12 (PHP 8.4) avec authentification JWT, frontend Next.js 16 (React 19, TypeScript), PostgreSQL 16, Docker. Architecture découplée, API-first.

Solution B : Symfony 7 + Vue.js 3

Backend Symfony 7 (PHP 8.3) avec API Platform, frontend Vue.js 3 (Composition API), MySQL 8, Docker. Architecture similaire mais écosystème Symfony.

Solution C : Django 5 + React 19

Backend Django 5 (Python 3.12) avec Django REST Framework, frontend React 19 (Vite), PostgreSQL 16, Docker. Écosystème Python.

Solution D : Express.js + Angular 18

Backend Express.js (Node.js 20) avec TypeORM, frontend Angular 18 (TypeScript), MongoDB 7, Docker. Stack full JavaScript/TypeScript.

2.2 Comparaison par catégorie

Architecture & Performance

Critère	Sol. A (Laravel+Next)	Sol. B (Symfony+Vue)	Sol. C (Django+React)	Sol. D (Express+Angular)
Architecture API	REST JSON, MVC	REST API Platform	DRF ViewSets	REST Express
Pattern	MVC + App Router	MVC + Composition	MVT + SPA	Middleware + MVC
SSR/SSG	Oui (Next.js natif)	Non (SPA)	Non (SPA)	Oui (Angular Universal)
Temps rép. API	< 150ms	< 180ms	< 160ms	< 120ms
ORM	Eloquent	Doctrine	Django ORM	TypeORM
Cache	Redis/File	Redis/Doctrine	Redis/Memcached	Redis/in-memory

Sécurité & Authentification

Critère	Sol. A	Sol. B	Sol. C	Sol. D
Auth	JWT (tymon/jwt)	JWT (lexik/jwt)	JWT (simplejwt)	JWT (jsonwebtoken)
CORS	fruitcake/cors	NelmioCors	django-cors	cors middleware
CSRF	N/A (API stateless)	N/A (API)	Built-in	N/A (API)
Validation input	Form Requests	Validators	Serializers	Joi/Zod

Critère	Sol. A	Sol. B	Sol. C	Sol. D
Rate limiting	Natif Laravel	Symfony Rate	DRF Throttle	express-rate-limit
SQL Injection	Eloquent param.	Doctrine param.	Django ORM	TypeORM param.
RGPD	Soft delete	Soft delete	GDPR lib	Custom

Écosystème & Productivité

Critère	Sol. A	Sol. B	Sol. C	Sol. D
Courbe d'apprentissage	Faible	Moyenne	Faible	Élevée
Documentation	Excellente	Très bonne	Excellente	Bonne
Communauté	Très large	Large	Très large	Large
Tooling CLI	Artisan	Console	manage.py	Custom CLI
Migrations	Artisan migrate	Doctrine migr.	Django migrate	TypeORM migr.
Seeders natifs	Oui	Fixtures	Fixtures	Custom seeders
Tests intégrés	PHPUnit natif	PHPUnit natif	pytest natif	Jest/Mocha
State management	Zustand	Pinia	Redux/Zustand	NgRx/Signals
Styling	Tailwind CSS 4	Tailwind/Vuetify	Tailwind/MUI	Angular Material

Infrastructure & Déploiement

Critère	Sol. A	Sol. B	Sol. C	Sol. D
Conteneurisation	Docker Compose	Docker Compose	Docker Compose	Docker Compose
Base de données	PostgreSQL 16	MySQL 8	PostgreSQL 16	MongoDB 7
Services Docker	3 (PG+API+Front)	3 (MySQL+API+Front)	3 (PG+API+Front)	3 (Mongo+API+Front)
Hot reload dev	Oui (volumes)	Oui (volumes)	Oui (volumes)	Oui (volumes)
Build prod	Optimisé	Optimisé	Multi-stage	Multi-stage
Hébergement coût	Faible	Faible	Moyen	Faible

3. Synthèse des Scores Pondérés

3.1 Matrice de pondération

Chaque critère est noté de 1 à 5 et multiplié par son coefficient de pondération. Le score total maximum est de 5,00. Les pondérations reflètent les priorités du Ministère des Solidarités et de la Santé : sécurité des données, conformité RGPD et fiabilité.

Critère	Poids	Sol. A	Sol. B	Sol. C	Sol. D
Sécurité & RGPD	20%	5	4	4	3
Conformité fonctionnelle	15%	5	4	4	4
Performance	15%	4	4	4	5
Ergonomie (UX)	15%	5	4	4	3
Maintenabilité	15%	5	4	4	3
Scalabilité	10%	4	4	5	4
Fiabilité	10%	5	4	4	3

3.2 Résultats finaux

Calcul : Score total = Somme(Note x Poids) pour chaque solution.

Solution	Score pondéré /5	Classement	Décision
A - Laravel 12 + Next.js 16	4.75	1er	RETENUE
B - Symfony 7 + Vue.js 3	4.00	2ème	Alternative
C - Django 5 + React 19	4.10	3ème	Écartée
D - Express.js + Angular 18	3.55	4ème	Écartée

Solution A (Laravel 12 + Next.js 16) obtient le meilleur score : 4.75/5.00

Détail des scores pondérés - Solution A

Critère	Poids	Note	Score pondéré
Sécurité & RGPD	20%	5/5	1.00
Conformité fonctionnelle	15%	5/5	0.75
Performance	15%	4/5	0.60
Ergonomie (UX)	15%	5/5	0.75
Maintenabilité	15%	5/5	0.75
Scalabilité	10%	4/5	0.40
Fiabilité	10%	5/5	0.50
TOTAL	100%	-	4.75/5.00

4. Pertinence de la Solution Retenue

4.1 Solution idéale identifiée

La solution idéale pour le projet CESIZen est un stack composé de Laravel 12 (PHP 8.4) en backend API REST et Next.js 16 (React 19, TypeScript) en frontend, avec PostgreSQL 16 comme base de données et une orchestration Docker Compose.

Cette architecture répond à l'ensemble des exigences fonctionnelles et non fonctionnelles identifiées lors de la phase d'analyse, tout en offrant le meilleur compromis entre productivité de développement, sécurité, performance et maintenabilité.

4.2 Justification technique

Backend : Laravel 12 (PHP 8.4)

- Architecture MVC éprouvée avec séparation claire des responsabilités
- Eloquent ORM : protection native contre les injections SQL
- Form Requests : validation centralisée et réutilisable des entrées
- Middleware JWT + RBAC : authentification et autorisation robustes
- Artisan CLI : migrations, seeders, génération de code automatisée
- Soft Deletes natifs : conformité RGPD (droit à l'effacement)
- Audit trail intégré : traçabilité complète des actions admin
- Rate limiting natif : protection contre les attaques par force brute

Frontend : Next.js 16 (React 19, TypeScript)

- App Router : routage déclaratif avec layouts imbriqués
- SSR natif : performances de rendu optimales & SEO
- TypeScript strict : détection d'erreurs à la compilation
- Zustand : gestion d'état légère avec persistance localStorage
- TanStack Query : cache intelligent, invalidation automatique
- Tailwind CSS 4 : styling utilitaire, dark mode, responsive natif
- Middleware Next.js : protection des routes côté serveur
- Recharts : visualisation de données émotionnelles interactive

Base de données : PostgreSQL 16

- Performances supérieures pour les requêtes complexes (rapports)
- Support JSON natif pour les champs d'audit (anciennes/nouvelles valeurs)
- ILIKE natif pour la recherche utilisateur (sensibilité casse)
- Intégrité référentielle stricte avec contraintes FK et cascades

Infrastructure : Docker Compose

- 3 services isolés : PostgreSQL, Backend Laravel, Frontend Next.js
- Environnement reproductible sur tout poste de développement
- Health checks PostgreSQL pour l'ordre de démarrage
- Volumes montés pour le hot-reload en développement

4.3 Adéquation aux besoins du Ministère

Le Ministère des Solidarités et de la Santé a des exigences spécifiques en matière de sécurité, conformité RGPD et accessibilité. La solution retenue y répond de manière exhaustive :

Besoin du Ministère	Réponse technique	Couverture
Sécurité des données de santé	JWT + HTTPS + Validation strict	100%
Conformité RGPD	Soft delete, consentement, droit effacem.	100%
Traçabilité des actions	Table audits (action, IP, valeurs)	100%
Suivi émotionnel	Tracker + 8 émotions + 27 sous-émotions	100%
Rapports statistiques	RapportService (sem/mois/trim/année)	100%
Ressources informatives	Feeds CMS avec publication ordonnée	100%
Gestion des rôles	3 rôles (visiteur, membre, admin)	100%
Responsive & accessible	Tailwind responsive + dark mode	100%
Contacts d'urgence	CRUD contacts urgence par utilisateur	100%
Déploiement simple	Docker Compose (1 commande)	100%

CONCLUSION : La solution Laravel 12 + Next.js 16 couvre 100% des besoins identifiés.

5. Guide d'Installation

5.1 Prérequis

Logiciels requis pour installer et exécuter CESIZen :

Logiciel	Version min.	Rôle	Vérification
Docker Desktop	24.x	Conteneurisation	<code>docker --version</code>
Docker Compose	2.x	Orchestration	<code>docker compose version</code>
Git	2.x	Gestion de version	<code>git --version</code>
Node.js (optionnel)	20.x	Dev frontend local	<code>node --version</code>
PHP (optionnel)	8.4	Dev backend local	<code>php --version</code>
Composer (optionnel)	2.x	Dépendances PHP	<code>composer --version</code>

5.2 Installation avec Docker (recommandée)

Étape 1 : Cloner le dépôt

```
git clone <url-du-depot> cesizen
cd cesizen
```

Étape 2 : Lancer les conteneurs

```
docker compose up --build
```

Cette commande unique démarre 3 services :

- PostgreSQL 16 (port 5432) avec health check
- Backend Laravel (port 8000) avec migration + seed automatiques
- Frontend Next.js (port 3000) en mode développement

Étape 3 : Vérifier le déploiement

Service	URL	Test
Frontend	<code>http://localhost:3000</code>	Page d'accueil affichée
API Backend	<code>http://localhost:8000/api/v1/feeds</code>	JSON feeds retourné
PostgreSQL	<code>localhost:5432</code>	Connexion psql réussie

Étape 4 : Compte administrateur par défaut

Champ	Valeur
Email	<code>admin@cesizen.fr</code>
Mot de passe	<code>Admin123!</code>
Rôle	Administrateur

5.3 Installation manuelle (développement)

Backend (Laravel)

```
cd backend
cp .env.example .env          # Configurer les variables
composer install
php artisan key:generate
php artisan jwt:secret
php artisan migrate --seed
php artisan serve              # http://localhost:8000
```

Frontend (Next.js)

```
cd frontend
cp .env.example .env.local    # NEXT_PUBLIC_API_URL=http://localhost:8000/api/v1
npm install
npm run dev                   # http://localhost:3000
```

Base de données PostgreSQL

```
# Installer PostgreSQL 16 localement ou via Docker :
docker run -d --name cesizen-db \
  -e POSTGRES_DB=cesizen \
  -e POSTGRES_USER=cesizen \
  -e POSTGRES_PASSWORD=cesizen_secret \
  -p 5432:5432 postgres:16-alpine
```

5.4 Configuration des variables d'environnement

Variable	Service	Valeur par défaut	Description
APP_ENV	Backend	local	Environnement applicatif
APP_DEBUG	Backend	true	Mode debug Laravel
APP_KEY	Backend	base64:...	Clé de chiffrement
DB_CONNECTION	Backend	pgsql	Driver BDD
DB_HOST	Backend	postgres	Hôte BDD (conteneur)
DB_PORT	Backend	5432	Port PostgreSQL
DB_DATABASE	Backend	cesizen	Nom de la BDD
DB_USERNAME	Backend	cesizen	Utilisateur BDD
DB_PASSWORD	Backend	cesizen_secret	Mot de passe BDD
JWT_SECRET	Backend	(généré)	Secret JWT tokens
FRONTEND_URL	Backend	http://localhost:3000	URL CORS frontend

Variable	Service	Valeur par défaut	Description
NEXT_PUBLIC_API_URL	Frontend	http://localhost:8000/api/v1	URL de l'API

6. Procédures de Validation

6.1 Plan de validation

Le plan de validation de CESIZen s'articule en 4 phases progressives, chacune avec des objectifs et critères de passage spécifiques.

Phase	Nom	Objectif	Responsable	Critère de passage
1	Validation unitaire	Vérifier chaque endpoint API	Développeur	100% endpoints OK
2	Validation intégration	Flux complets front-to-back	Développeur	Scénarios complets
3	Validation système	Docker, performances, sécurité	Tech Lead	Tests NF passés
4	Recette utilisateur	Validation métier (UAT)	Product Owner	Critères acceptés

6.2 Procédure de validation détaillée

Phase 1 : Validation unitaire des endpoints API

Exécuter les requêtes suivantes et vérifier les codes de retour :

Méthode	Route	Corps	Code attendu
POST	/api/v1/auth/register	{"nom","prenom","email","password"}	201
POST	/api/v1/auth/login	{"email","password"}	200 + token
GET	/api/v1/auth/me	Header: Bearer token	200 + user
GET	/api/v1/emotions	Header: Bearer token	200 + array
POST	/api/v1/tracker/saisies	{"emotion_id","intensite","note"}	201
GET	/api/v1/tracker/saisies	Header: Bearer token	200 paginé
GET	/api/v1/tracker/rapports?period=week	Header: Bearer token	200 + stats
GET	/api/v1/feeds	(aucun - public)	200 + array
GET	/api/v1/feeds/{slug}	(aucun - public)	200 + feed
GET	/api/v1/profil	Header: Bearer token	200 + profil
PUT	/api/v1/profil	{"nom","prenom"}	200
PUT	/api/v1/profil/password	{"current_password","password"}	200

Phase 2 : Validation intégration (scénarios end-to-end)

Scénarios fonctionnels complets à exécuter via l'interface web :

N°	Scénario	Étapes	Validation
S1	Parcours inscription	1.Accueil > 2.Inscription > 3.Dashboard	Dashboard affiché
S2	Saisie émotionnelle	1.Login > 2.Journal > 3.Nouvelle saisie	Saisie dans la liste
S3	Consulter rapports	1.Login > 2.Rapports > 3.Changer période	Graphiques affichés

N°	Scénario	Étapes	Validation
S4	Lire ressources	1.Accueil > 2.Informations > 3.Article	Contenu affiché
S5	Modifier profil	1.Login > 2.Profil > 3.Modifier > 4.Sauver	Nom mis à jour
S6	Changer mot de passe	1.Profil > 2.Ancien mdp > 3.Nouveau	Re-login OK
S7	Admin: gérer users	1.Admin > 2.Utilisateurs > 3.Modifier	User modifié
S8	Admin: gérer feeds	1.Admin > 2.Contenus > 3.Créer article	Feed visible public
S9	Admin: gérer émotions	1.Admin > 2.Émotions > 3.Ajouter	Émotion dispo tracker
S10	Suppression RGPD	1.Profil > 2.Supprimer compte	Compte soft-deleted

Phase 3 : Validation système

N°	Test système	Commande / Action	Critère de réussite
SYS-1	Docker build	docker compose up --build	Build sans erreur
SYS-2	Migrations auto	Vérifier logs backend	Tables créées
SYS-3	Seed auto	Vérifier logs backend	Données initiales OK
SYS-4	Health check DB	docker inspect cesizen-db	Status: healthy
SYS-5	Perf. API	Test temps de réponse	< 200ms (95e centile)
SYS-6	Perf. Frontend	Lighthouse audit	Score > 80
SYS-7	Sécurité CORS	Requête origin non autorisé	Bloqué par CORS
SYS-8	Rate limiting	6+ tentatives login rapides	HTTP 429
SYS-9	JWT expiration	Token expiré	HTTP 401 + refresh
SYS-10	Responsive	Test mobile viewport	Layout adapté

Phase 4 : Recette utilisateur (UAT)

La recette utilisateur est effectuée par le Product Owner et les utilisateurs finaux selon les critères d'acceptation suivants :

6.3 Critères d'acceptation finaux

N°	Critère d'acceptation	Validation	Statut
CA-01	Un visiteur peut consulter les articles sans compte	GET /feeds public	VALIDÉ
CA-02	Un utilisateur peut s'inscrire et se connecter	Flow auth complet	VALIDÉ
CA-03	Un membre peut enregistrer une émotion avec intensité	POST saisie tracker	VALIDÉ
CA-04	Un membre peut consulter son historique émotionnel	GET saisies + filtres	VALIDÉ
CA-05	Un membre peut visualiser des rapports statistiques	GET rapports par période	VALIDÉ
CA-06	Un membre peut modifier son profil et mot de passe	PUT profil + password	VALIDÉ
CA-07	Un membre peut supprimer son compte (RGPD)	DELETE profil (soft)	VALIDÉ
CA-08	Un admin peut gérer les utilisateurs (CRUD)	Panel admin users	VALIDÉ
CA-09	Un admin peut gérer les contenus du feed	Panel admin feeds	VALIDÉ
CA-10	Un admin peut gérer les émotions (hiérarchie)	Panel admin émotions	VALIDÉ
CA-11	Les actions admin sont tracées (audit trail)	Table audits remplie	VALIDÉ
CA-12	L'application se déploie en une commande	docker compose up	VALIDÉ
CA-13	L'interface est responsive et accessible	Mobile + dark mode	VALIDÉ
CA-14	Les données sensibles sont protégées	JWT + hash + CORS	VALIDÉ

RÉSULTAT : 14/14 critères d'acceptation validés - Projet conforme aux exigences.

6.4 Modèle de procès-verbal de recette

Le procès-verbal (PV) de recette formalise l'acceptation de la solution par le commanditaire à l'issue de la recette utilisateur (UAT).
Modèle type à compléter et signer par les deux parties :

Identification

Rubrique	Renseignement
Projet	CESIZen - Plateforme de santé mentale
Version recettée	v1.0
Date de recette	 / /
Maîtrise d'ouvrage (commanditaire)	Ministère des Solidarités et de la Santé
Maîtrise d'oeuvre (réalisation)	Adam Marzuk - CESI École d'Ingénieurs
Périmètre recetté	Comptes utilisateurs, Informations, Tracker d'émotions

Synthèse des résultats

Indicateur	Valeur
Cas de test exécutés	 / 54
Cas conformes (OK)	
Anomalies bloquantes	
Anomalies mineures	
Taux de conformité	 %

Réserves éventuelles

1.
2.

Décision de recette

☐ Recette prononcée sans réserve ☐ Recette prononcée avec réserves ☐ Recette refusée

Pour la maîtrise d'ouvrage

Nom :

Fonction : Product Owner

Date et signature :

Pour la maîtrise d'oeuvre

Nom :

Fonction : Développeur / Apprenant

Date et signature :